

ONYX Supervisor

The mission coordinator. Turns a business goal into a bounded plan, assigns the right specialists, joins their evidence and presents one understandable result.

Who this is for: product, engineering, operations and anyone who has to explain XELOR to somebody else.

What it covers: the work ONYX reasons about, the rules it cannot break, the exact tools it can call, and what is honestly not built yet.

Truth standard: the repository as it stands on 7 August 2026.

ONYX

What ONYX is for

A role with a fixed tool list, not a general assistant

In one sentence

Turns a business goal into a bounded plan, assigns the right specialists, joins their evidence and presents one understandable result.

3

business areas it speaks for

1

tools it can actually call

3

capability families allowed

8

agents it may delegate to

What reaches it

A goal, a Decision Commander risk or an ERP signal ·
Evidence returned by every specialist · HEXA's verification
result · The human approval decision

What it hands back

A scoped, versioned mission run · A joined evidence pack · A
coordinated plan · A final outcome naming its own limits

Capability families it is allowed to touch

agent.

workflow.

general.

Ownership and authority are not the same thing

The areas above are where ONYX is the right specialist to ask. The tool table on page 8 is the much smaller set it can invoke at runtime. Owning a business area does not grant runtime power over it — and for ONYX, the supervisor, the gap between the two is the widest in the system.

What sits behind ONYX

Records, screens, workflow, controls and hand-offs

ONYX Copilot		ONYX module · copilot
Purpose	A read-only question surface for factory data. It gives a short answer, names the evidence rows and refuses questions outside its closed catalogue.	
Core records	21 registered intents; question, outcome, matched intent, row count, sources, correlation id and optional narration result.	
User surface	Ask; Question log. Main API surfaces: /copilot/capabilities, /copilot/ask and /copilot/history.	
End-to-end flow	Rules match the question first; a model may only help choose among known intents. The user's permission is checked, one hand-written query runs in the tenant transaction, a deterministic answer is rendered, optional narration is provenance-checked, and the event is logged.	
Enforced controls	No SQL generation, no write tool, per-intent permission and row cap, unknown-intent refusal, numeric provenance check, deterministic fallback and an auditable refusal path.	
Inputs and outputs	Reads Sales, Purchase, Inventory, Planning, Production, Quality, Maintenance, Accounts and master-data views through fixed queries; it never owns those records.	
Current boundary	The 21 questions are live. General conversation and unrestricted business advice are not; external-model narration is off by default.	
Primary code map	apps/web/src/modules/copilot/manifest.ts · apps/api/src/modules/copilot/	

Agent OS		ONYX module · agentos
Purpose	Durable coordination for cross-functional decisions: it runs versioned mission graphs, gathers specialist evidence and pauses at human approval gates.	
Core records	Agent definitions, capabilities, graph versions and hashes, runs, node attempts, events, checkpoints, approvals, signals, outcomes and governed work items.	
User surface	Decision Commander; Human Approvals; Mission Command. Catalogue, run, signal, approval, action, memory, knowledge-graph and readiness APIs.	
End-to-end flow	Start from a person, signal or risk; freeze the graph and authority context; run all ready nodes in parallel waves; checkpoint each wave; join evidence; let HEXA verify; wait for the named approver; resume; then publish or dispatch only what the graph permits.	
Enforced controls	Fixed tool allow-lists, user-permission intersection, maximum steps and timeouts, bounded retries, content-hashed graphs, resumable checkpoints and engine-enforced approval ancestry for every effect.	
Inputs and outputs	Coordinates all eight specialists. Domain reads still pass through their normal services, and the single effect writes a governed assignment rather than modifying ERP records. ACHILES has no effectful tool.	
Current boundary	Six mission graphs and durable orchestration are live. Agent language reasoning is deterministic and dispatched work remains work for a person, not proof of completion.	
Primary code map	apps/web/src/modules/agentos/manifest.ts · apps/api/src/agent-os/	

How to read these module pages

“Live” means the records, service rules and user/API surface exist in this repository. It does not mean every surrounding enterprise process is complete. The boundary row names what remains illustrative, manual, simulate-driven or outside the MVP.

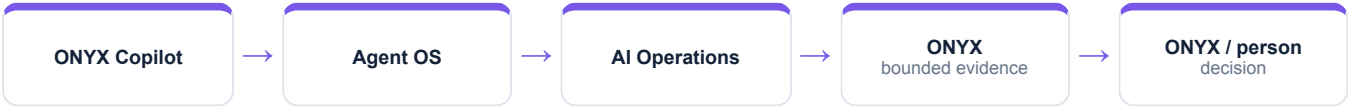
How ONYX's modules connect

A module owns its records; the agent explains the combined evidence

AI Operations		ONYX module · aiops
Purpose	The control room for AI features: what is registered, which provider may serve it, what it cost, whether it passed evaluation and when a person must review it.	
Core records	Feature registry, provider configuration, governance state, tenant opt-outs, budgets, usage/cost events, evaluations, review items, incidents and a hash-chained AI action log.	
User surface	Connectors; Feature registry; Providers; Evaluations; Cost; Human review; Incidents.	
End-to-end flow	A caller names a registered feature; the router checks global and tenant controls, budget and provider policy; the provider or deterministic fallback runs; input/output metadata and cost are logged; evaluation and human feedback determine whether the feature remains eligible.	
Enforced controls	Closed registry, kill switch, tenant opt-out, budget ceiling, no-ship evaluation gate, privacy mode, deterministic fallback and tamper-evident action logging.	
Inputs and outputs	Supplies governance to Copilot and the domain AI features in Employee Spend, Customer Care & Warranty, HR and master data; receives outcomes and edits for later evaluation.	
Current boundary	Governance, registry, cost and evaluation plumbing are live. Provider connectivity is not configured in the demo, and some registered features remain unimplemented or lack golden sets.	
Primary code map	apps/web/src/modules/aiops/manifest.ts · apps/api/src/modules/aiops/	

Cross-module coordination

Copilot handles one bounded question; Agent OS handles a cross-functional mission; AI Operations governs any model-assisted feature. They share identity, permissions, logging and refusal rules, but they do not share a hidden general-purpose action channel.



How ONYX's world actually runs

The business pipeline, not the agent plumbing

ONYX has two surfaces. The Copilot answers one question from one module. Agent OS runs a mission across many. Both refuse in the same way: there is no endpoint that takes free text and runs it.

1

Understand

Rules first. A model is asked only when the rules cannot decide, and its answer is checked against the closed list of 21 intents before it counts.

2

Authorise

The matched question declares a permission and the asker must hold it — the same check the equivalent screen makes, so the Copilot is never a side door.

3

Retrieve

One hand-written SELECT, on the caller's own transaction, under the caller's own tenant. The model is not in this step and cannot reach it.

4

Render

From a template. Numbers on screen are numbers from the rows.

5

Narrate

Optional and OFF by default. A model may re-word the answer and is held to numeric provenance; if it fails, the plain answer is shown.

6

Log

Every question, answered or refused, with what it read and how many rows.

How much of this ONYX can actually see

Of everything above, ONYX can read exactly one thing directly — company masters. The rest of this pipeline is context for judging what it reads; it is not data the agent can reach. Where a mission needs more, it asks the specialist who owns it or it says the evidence is missing.

What the code will not let anyone do

Enforced by constraints and locks, not by wording

The coordinator is the least powerful agent

ONYX holds one read capability and is NOT on the allow-list for `agent.action.dispatch`. It can convene the mission; it cannot act in the business.

The registry is closed

A call for an AI feature key outside the registered set is rejected at the router with `AI_FEATURE_NOT_REGISTERED` before a token is spent.

Refusal is auditable

A governance refusal is itself logged to the hash-chained AI action log, then fails closed so the caller drops to its deterministic mode.

Narration cannot invent a number

Every digit in a narrated answer must trace to a retrieved row, or the template answer is shown instead.

Why this page matters more than the agent pages

An agent is only as trustworthy as the system it reads. Every rule here holds whether the request came from a person, a screen, a seeder or ONYX — which is precisely why an agent can be given a read tool without also being given a way to do damage.

Where these rules are kept

`apps/api/src/agent-os/agent-graph.engine.ts`

`apps/api/src/modules/copilot/copilot.service.ts`

`packages/platform/src/ai/feature-registry.ts`

How ONYX takes part in a mission

The engine controls order, parallelism and recovery



1 · Scope

ONYX turns the goal into a run against a frozen graph version, with a fixed tenant, user, step budget and timeout.

2 · Authorise

Two checks, both required: the tool must be on ONYX's allow-list, and the signed-in person must independently hold the permission.

3 · Execute

A registered domain service does the work under the same rules a screen would face. There is no general SQL doorway.

4 · Preserve

Inputs, outputs, events and a checkpoint after every wave, so the run can be explained or resumed.

An agent can narrow authority, never widen it

ONYX may delegate only to HEXA, MICA, SPAR, AXLE, KILN, RASP, RELAY, ACHILES. Because the permission check is repeated against the requesting person, a mission can never reach data that person could not have opened themselves.

Everything ONYX can call

This table is the complete list — there is no other door

Capability	Mode	What comes back	Permission required	Needs approval
general.companies.read	Read	Company masters	general.company.read	No

Read · Analyse · Simulate

Return evidence or a reproducible view. Nothing in the business changes.

Draft

Produce reviewable content that is labelled a draft and can never present itself as an approved outcome.

Execute

The only side-effecting mode in the whole system, and the only one that requires an approved human gate above it.

Both locks must open

The agent's allow-list AND the person's permission. Either one failing is a refusal.

ONYX cannot act in the business at all

There is no execute capability on this list, and ONYX is not on the allow-list for the one that exists. The agent that coordinates every mission is the only one that cannot cause an effect — which is the point, not an oversight.

The Copilot's separate surface — 21 questions, and no twenty-second

A mission is not the only way in. The Copilot answers a fixed list of questions, each tied to one hand-written query and one permission. Anything outside the list is refused rather than improvised.

Area	Questions	Examples of what can be asked
Planning	4	What to buy, what to make, what is past due, where the shortages are
Stock	3	On hand, by warehouse, recent movements
Sales	3	Open orders, one order's status, what is due soon
Purchase	3	Open orders, one order's status, what is awaiting receipt
Master data	3	Find an item, a vendor or a customer
Production	2	Open orders, one order's status
Quality · Maintenance · Accounts	3	Pending inspections, open work orders, what is outstanding

Where ONYX actually appears

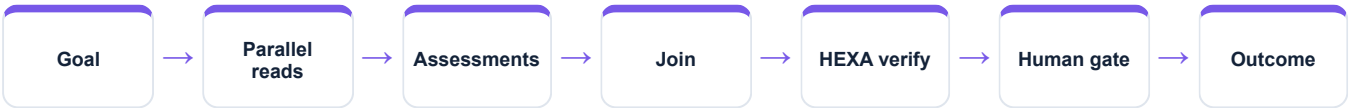
Node by node, from the registered graph definitions

Mission graph	What ONYX does in it
Cross-functional readiness	Frames the mission, then writes the final synthesis after approval
Nine-agent operating review	Frames it, then issues the command brief
Controlled action mission	Frames it, builds the action plan, publishes the outcome
Working Capital Review	Sets the horizon, then publishes the brief
QMS & Audit Readiness	Sets the audit scope, then publishes gaps and owners
Managed Service Assurance	Frames the customer outcome and decisions reserved for people

Reading this honestly

“Not involved” is not a limitation, it is the design. A mission asks only the specialists whose evidence it needs, and a specialist cannot add itself to a graph at runtime. The graph is written down, versioned and content-hashed before the run starts.

The shape every mission shares



“Should we ship the twelve held pumps anyway to make the date?”

Real records from the running demo, not an illustration

The Copilot refuses. This is not a judgement the model got right — there is no endpoint that takes a question and runs it, none that takes SQL, and none that writes. The read-only promise is kept by there being nothing to call.

Asked instead for what it does hold — “how much PMP-PX400 do we have” — it answers from stock_balance, item and warehouse, and cites all three.

For the cross-functional version of the question, ONYX starts a mission instead: the relevant specialists read their own evidence in parallel, HEXA verifies, a person approves, and only then is a brief published.

The sentence to say out loud

“ONYX contributes evidence from its own area, with the source records behind it and its limits stated. It does not claim an action is done until the system has recorded and verified it.”

Fair questions to ask while watching

- Which records is this answer standing on?
- Which permission was checked, and against whom?
- Is this a recorded fact, a simulation, a draft, or a completed result?
- Would this step have waited for a person?
- What happens if the service restarts halfway through?

Live today, and deliberately not

The second column is the one worth reading

Working now • 21 permission-checked Copilot questions • 6 registered mission graphs • Parallel waves, checkpoints and recovery • Signal and Decision Commander mission triggers • Closed AI registry with kill switch and cost ledger	Not built — stated plainly • Language reasoning is deterministic; no external model API is active • ONYX cannot dispatch an action — it is not on the allow-list • Narration is off by default • The Copilot answers 21 questions, not any question
--	--

Identity boundary	Verified token → tenant and actor
Authority boundary	Agent allow-list AND the person's permission
Action boundary	Side effects need an approved ancestor node
Data boundary	Domain service over a tenant-fenced database
Evidence boundary	Append-only runs, events, checkpoints and audit

Gaps are listed because a guide that hides them fails the first hour of technical due diligence. Every item on the right is either a roadmap decision or a known defect, and both are recorded in the project's own capability-gap register.

ONYX on one page

For explaining the agent to somebody new

Its job

Turns a business goal into a bounded plan, assigns the right specialists, joins their evidence and presents one understandable result.

Speaks for

ONYX Copilot, Agent OS, AI Operations

Takes in

A goal, a Decision Commander risk or an ERP signal · Evidence returned by every specialist · HEXA's verification result · The human approval decision

Gives back

A scoped, versioned mission run · A joined evidence pack · A coordinated plan · A final outcome naming its own limits

Can call

general.companies.read

Cannot do

Language reasoning is deterministic; no external model API is active · ONYX cannot dispatch an action — it is not on the allow-list · Narration is off by default

Words used on these pages

Agent	A named specialist with a fixed, allow-listed set of tools — not a process that can roam.
Capability	One registered operation with a declared mode, owner and required permission.
Mission graph	A versioned recipe: which steps run, which run together, where it waits for a person.
Checkpoint	A stored snapshot after each wave, used to explain a run and to resume it safely.
Governed work item	An approved, append-only assignment for a human team — not the business change itself.